PBL Milestone 1: Algorithm Architecture Specification & Software Engineering Proposal

Drone Defense

Balase, Mark Nathaniel

Canapi, Seanne Dominique F.

Gelindon, Justine Clein

Gensaya, Lorens Mark

Grande, Jeremy

11/04/2026

Metadata Field	Project Team Specification
Project Title	Drone Defense
Team Name	3GBC
Section & Year	III-CCSAD
Team Members & Roles	Balase, Mark Nathaniel (*UX Designer/Programmer*) Canapi, Seanne Dominique F. (*UX/UI Designer/Programmer*) Gelindon, Justine Clein (*Project Manager*) Gensaya, Lorens Mark (*Artist/Programmer*) Grande, Jeremy (*Artist*)
Repository URL	https://github.com/bosic2/AlgoCom.git
Submission Date	11/4/2026

<AI Assistance and Transparency: We asked Google's AI to assist in explaining Task 4, outlining pseudocode in Task 2, explaining certain parts of the chosen algorithms, and explaining some of the instructions in the template document.>

Task 1: Computational Problem Modeling & Formal Mathematical Formulation

Problem 1: Dynamic Difficulty

Problem Statement

Dynamically adjusting difficulty levels based on player performance, checking for the base's health to dynamically adjust enemy spawns for the next incoming subwave

Input Parameters:

H_{base} = health of base

H_{max} = maximum health of base

E_{active} = active enemies in subwave

$E_{subwave}$ = total enemies in subwave

E_{budget} = enemy budget for subwave

dt = time elapsed since start of wave

K_p, K_i, K_d = control loop coefficients

Boundary Conditions & Constraints:

1. If t = 0, difficulty multiplier is equal to 1, keeping the difficulty at a neutral state
2. If $H_{base} < (H_{max} * 0.25)$, difficulty modifier will be hard limited to 75%.
3. If $E_{active} < (E_{subwave} * 0.5)$, next subwave will spawn earlier
4. Difficulty (d) is limited to $(0.75 < d < 1.2)$.

Objective Function: Optimization Problem

Output Certificate:

The algorithm outputs the next subwave's enemy budget for the next incoming subwave based on the factors such as player base health and number of enemies active.

Problem 2: Upgrade System

Problem Statement

Building a simple upgrade system that allows for upgrade paths.

Input Parameters:

V = upgrade nodes for player and allies

E = prerequisite edges between upgrades

G = (V, E)

Boundary Conditions & Constraints:

1. The upgrade system must strictly be in the form of an Adjacency List DAG to prevent upgrades requiring themselves.
2. Upgrades must have all prerequisites complete before being accessible

Objective Function: Classification Problem

Output Certificate:

The algorithm outputs a full sorted upgrade path (DAG) filled with upgrade nodes as vectors and prerequisites as edges.

Task 2: Algorithmic Paradigm Selection & Logic Specification

Problem 1: Dynamic Difficulty

Schema Field	Team Engineering Details
Algorithm Name	PID Controller
Design Paradigm	Control Theory
Foundational References	Aström, K. J., & Hägglund, T. (2006). Advanced PID Control. Hunicke, R. (2005). AI for Dynamic Difficulty Adjustment. Proceedings of the GDC AI Wisdom Workshop.
Baseline Alternative	Finite State Machine
Computational Justification	Best, Average, and Worst Time Complexity is O(1), and Space Complexity is O(1), while being more complex and dynamic than the baseline alternative.
Invariants & Recurrence Relations	1. $0.75 < d < 1.2$ 2. if ($H_{base} < H_{max} * 0.25$), then $d = 0.75$

Pseudocode:

```
kp = proportional_gain
ki = integral_gain
kd = derivative_gain

integral_sum = 0
previous_error = 0
error = 0
last_time = get_current_time()

function calculatePID(target_setpoint, current_measured_value):
    current_time = get_current_time()
    dt = current_time - last_time

    # prevents division by zero
    if dt <= 0:
        dt = 0.001

     # calculating error
     error = target_setpoint - current_measured_value
```

```
    P = kp * error

    integral_sum = integral_sum + (error * dt)
    I = ki * integral_sum

    derivative = (error - previous_error) / dt
    D = kd * derivative

    output = P + I + D

    previous_error = error
    last_time = current_time

    return output
```

Problem 2: Upgrade System

Schema Field	Team Engineering Details
Algorithm Name	Kahn's Algorithm
Design Paradigm	Graph Theory
Foundational References	Kahn, A. B. (1962). Topological sorting of large networks.
Baseline Alternative	Depth-First Search (DFS)
Computational Justification	Time complexity is O(V+E) while Space complexity is O(V), and strictly limits it to a DAG, thus disallowing cyclical errors.
Invariants & Recurrence Relations	1. A node is accessible if indegree == 0 2. T(V, E) = O(V + E)

Pseudocode

```
function kahnsAlgorithm(graph):
    inDegree = array or map initialized to 0 for all nodes
    queue = empty queue
    topologicalOrder = empty list

    for each node in graph:
        for each neighbor of node:
        inDegree[neighbor] += 1
```

```
for each node in graph:
    if inDegree[node] == 0:
        queue.enqueue(node)

while queue is not empty:
    currentNode = queue.dequeue()
        topologicalOrder.append(currentNode)

for each neighbor of currentNode:
    inDegree[neighbor] -= 1

    if inDegree[neighbor] == 0:
        queue.enqueue(neighbor)

if length(topologicalOrder) != total_number_of_nodes:
    return "Error: Graph contains a cycle (Not a DAG)"

return topologicalOrder
```

Task 3: Theoretical Asymptotic Analysis & Scalability Profiling

Asymptotic Complexity Derivations (PID Controller)

Worst-Case Time Complexity: O(1)

Average/Best-Case Time Complexity: O(1)

Auxiliary Space Complexity: O(1)

Big Data Dimension Evaluation (PID Controller)

Dimension	Engineering Parameter	System Scalability Impact & Mitigation Strategy
Volume	O(1) time and space complexity; only tracks current error, accumulated integral, and previous error floats.	As enemy count increases, the PID controller must count more enemies; however, this is negligible and a mitigation strategy is not needed, as the only way to reduce impact is to reduce enemy count anyways. Have a separate object manage and keep track of enemy counts.
Velocity	Executes in a 60 frames per second;	May cause lag spikes or slower framerate;

	called every frame	mitigation strategy is to instead lower the rate of PID controller updates to a fixed time interval to reduce number of execution
Variety	Handles numerical floating-point values such as player base health and enemy counts	Mismatching data types such as float and int may break PID; mitigation strategy is to normalize the values to a single floating-point number for the algorithm to understand

Asymptotic Complexity Derivations (Kahn's Algorithm)

Worst-Case Time Complexity: O(V + E)

Average/Best-Case Time Complexity: O(V + E)

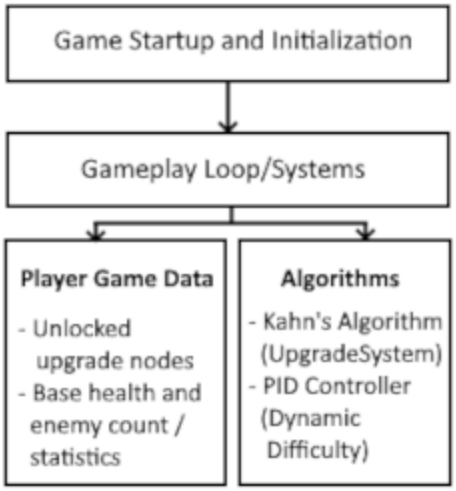
Auxiliary Space Complexity: O(V)

Big Data Dimension Evaluation (Kahn's Algorithm**)**

Dimension	Engineering Parameter	System Scalability Impact & Mitigation Strategy
Volume	V = how many upgrade nodes we are including; E = how many prerequisites there are in total for every node.	High memory if DAG is traversed raw; mitigation strategy is to track player progress via a localized hash table instead of mutating the entire graph.
Velocity	Called upon initializing the upgrade menu after wave finishes.	Only runs whenever called after the wave; however, mitigation strategy is to unlock upgrades asynchronously during the wave's downtime even if the upgrade system is not visible
Variety	Nodes contain multiple variables of different datatypes such as strings for names and descriptions, integers for level requirements, floats for upgrade multipliers, etc	Too much variety in data types may slow down execution speed; mitigation strategy is to use a primitive integer array, and store information in a separate dictionary.

Task 4: Decoupled Software Architecture & Service-Layer Integration

Architecture Component Diagram



JSON Request (PID Controller)

```
{
“base_current_health”: 0.75,
“base_max_health”: 1.0,
“enemies_active”: 5,
“enemies_spawned_in_subwave”: 8
“delta_time”: 2.0
}
```

JSON Result (PID Controller)

```
{
  "next_subwave_spawn_budget": 42
}
```

JSON Request (Kahn’s Algorithm)

```
{
“total_tree_nodes”: 4,
“prerequisite_pairs”: [
{“from”: 0, “to”: 1},
{“from”: 1, “to”: 2},
{“from”: 2, “to”: 3}
],
"player_unlocked_ids": [0]
}
```

JSON Result (Kahn’s Algorithm)

```
{
  "full_path":,
```

```
  "immediately_available_unlocks":,
  "cycle_detected_error": false
}
```

Task 5: Agile Implementation Backlog & Empirical Validation Plan

User Stories (PID Controller)

1. As a player, I want the game's difficulty to dynamically adjust itself seamlessly while I play so that my experience is immersive and fair.

2. As a developer, I want the game's difficulty to be adjustable based on the game's activity so that I do not have to manually adjust it myself. Instead, I will only change the parameters of this algorithm to fine-tune the difficulty boundaries and curve.

Engineering Implementation & Benchmarking Ticket (PID Controller)

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-101
Module/Service	scripts/DynamicDifficulty.cs
Associated Algorithm	PID Controller
Edge-Case & Security Defenses	1. Forces dt to a minimum of 0.001 to prevent division by zero
Concrete Implementation Steps	1. Zero-initialize integral_sum, previous_error, error, and last_time 2. Calculate the elapsed time (dt) since the last check by subtracting last_time from the current time. 3. Force dt to a minimum of 0.001 if it is zero or negative to keep the code from crashing. 4. Find the game difficulty error by subtracting the current player/game tension score from the target difficulty setpoint. 5. Multiply this difficulty error by kp to get the instant reaction value (P). 6. Add the current difficulty error scaled by dt directly into the running history tracker (integral_sum). 7. Multiply that history tracker by ki to get the steady-state correction value (I). 8. Subtract previous_error from the current error and divide by dt to figure out how fast the player's situation is changing. 9. Multiply that rate of change by kd to get the predictive damping value (D). 10. Add P, I, and D together to get the final difficulty output, which determines a new subwave spawning budget.

	11. Save the current difficulty error into previous_error and the current timestamp into last_time to prepare for the next wave check.
Automated Benchmarking Assertions	Runtime per call = 0.34 microseconds with 30,000 iterations

User Stories (Kahn's Algorithm)

1. As a player, I want a responsive upgrade system that works functionally and intuitively, and will update as the game progresses with clear prerequisites and conditions.

2. As a developer, I want the upgrade system to be easy to build so that implementing more features and upgrades will be far more convenient in the future.

Engineering Implementation & Benchmarking Ticket (Kahn's Algorithm)

Implementation Field	Specification Detail
Ticket ID	DEV-ALGO-102
Module/Service	scripts/UpgradeSystem.cs
Associated Algorithm	Kahn's Algorithm for Directed Acyclic Graph (DAG)
Edge-Case & Security Defenses	1. Returns an empty list if no upgrades are configured in order to prevent index errors. 2. Detects impossible cyclical loops and throws an error if an upgrade requires itself in any way.
Concrete Implementation Steps	1. Create tracking list (inDegree) set to 0 for all upgrade options, an empty processing queue, and an empty path list (topologicalOrder). 2. Read through the entire tech tree graph and count how many prerequisite upgrades each individual node requires, saving these totals in the tracking list. 3. Look through all available upgrades in the graph and add any node that has zero prerequisite requirements (inDegree == 0) directly into queue. 4. Pull the first available node out of the queue (currentNode) and add it to the valid path list (topologicalOrder). 5. Look at all the downstream upgrades that depend on this processed node and reduce their dependency prerequisite counts by 1. 6. If any downstream upgrade's prerequisite count drops exactly to 0, immediately add it to the queue so it can be processed next. 7. Keep looping through steps 4, 5, and 6 the queue until processing queue runs completely empty. 8. Compare the final count of your path list against the total number of items in the database to check for loops. 9. Block the system and throw a cycle error if the numbers don't

	match, safely revealing a broken circular dependency loop. 10. Return finalized path list to safely display the unlocked, unblocked upgrade options to the player.
Automated Benchmarking Assertions	Runtime = 11ms with 1000 graph nodes

Task 6: Academic Review Questions & Defense Checkpoints

1. Why must heavy algorithmic processing be decoupled from database ORMs and web request threads?
    - This is to prevent resource starvation, which causes freezes for other incoming requests, and to avoid the massive CPU/memory overhead of ORM object hydration. Heavy processing should be handled by asynchronous background workers. (Fowler, 2002)

2. If your proposed algorithm has an exponential worst-case complexity ($O(2^n)$ or $O(n!)$), what heuristic, pruning, or approximation technique will maintain production viability?
    - Using optimization techniques like Simulated Annealing (Kirkpatrick et al., 1983) provides a less severe worst-case complexity for a “good enough” result, which is more efficient for large systems than using exact algorithms that are more accurate but slower.

3. What is the operational difference between an **Exact Algorithm** and an **Approximate / Heuristic Algorithm** in production optimization?
    - An exact algorithm is slower and more expensive but is more guaranteed to find the true optimum. An approximate/heuristic algorithm is faster and cheaper than an exact algorithm, but its results are slightly less accurate and not always 100% correct. (Vazirani, 2001)

4. How do you distinguish between an empirical benchmarking profiling approach (post-implementation) and an asymptotic theoretical Big-O analysis (pre-implementation)?
    - The benchmarking profiling approach requires actual code, while Big-O analysis only needs pseudocode. The former is for testing the actual algorithm’s results and runtime, while the latter is for predicting the time and space complexity of your chosen algorithm. (Knuth, 1997)

5. If your project utilizes machine learning or user-preference matrices, how will you mitigate the **Cold Start Problem** or **Data Sparsity Problem** during initial launch?

- To solve the Cold Start Problem, using Content-Based Filtering (IBM, 2024) by using the items' information and metadata to best match the user's preferences is an efficient solution to prevent this issue.

6. How does your algorithm handle edge-case inputs (e.g., disconnected subgraphs, singular matrices, cyclic dependencies) without crashing the host service?
    - Kahn's Algorithm returns an empty list if no upgrades are configured in order to prevent index errors, and will detect if any nodes (upgrades) require themselves. The system will throw a CyclicDependencyException or use an empty collection to keep the application loop running safely. (Kahn, 1962)

References

Aström, K. J., & Hägglund, T. (2005). *Advanced PID control*. John Wiley & Sons.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Hunicke, R. (2005, June). The case for dynamic difficulty adjustment in games. *In Proceedings of the 2005 ACM SIGCHI International Conference on Advances in computer entertainment technology* (pp. 429-433).

IBM. (2024, November 27). *What is content-based filtering?*. https://www.ibm.com/think/topics/content-based-filtering

Kahn, A. B. (1962). Topological sorting of large networks. *Communications of the ACM*, 5(11), 558-562.

Kirkpatrick, S. (1984). Optimization by simulated annealing: Quantitative studies. *Journal of statistical physics,* 34(5), 975-986.

Vazirani, V. V. (2001). *Approximation Algorithms*. Springer-Verlag.

III-CCSAD ALGOCOM

DRONE DEFENSE

PROBLEMS AND ALGORITHMS

Problem 1: Dynamic Difficulty

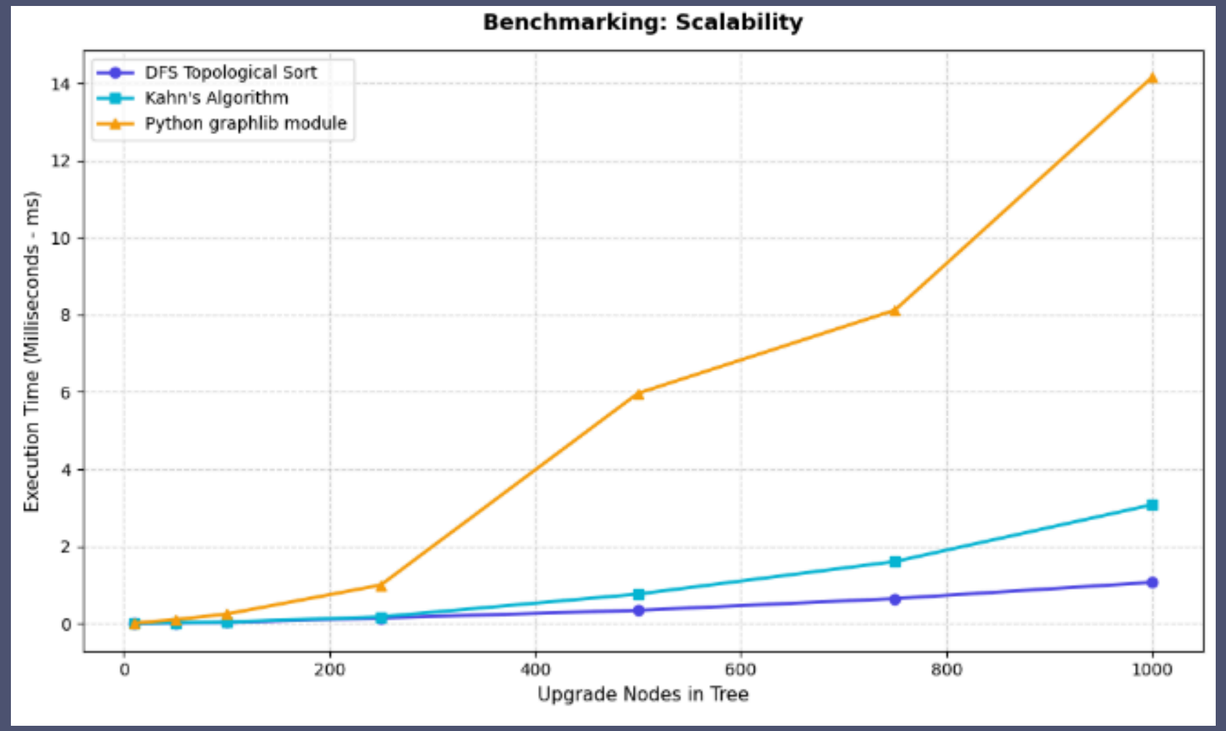
Dynamically adjusting difficulty levels based on player performance, checking for the base's health to dynamically adjust enemy spawns for the next incoming subwave

Algorithm: PID Controller

Problem 2: Upgrade System

Building a simple upgrade system that allows for upgrade paths, which is both intuitive and convenient for players and developers.

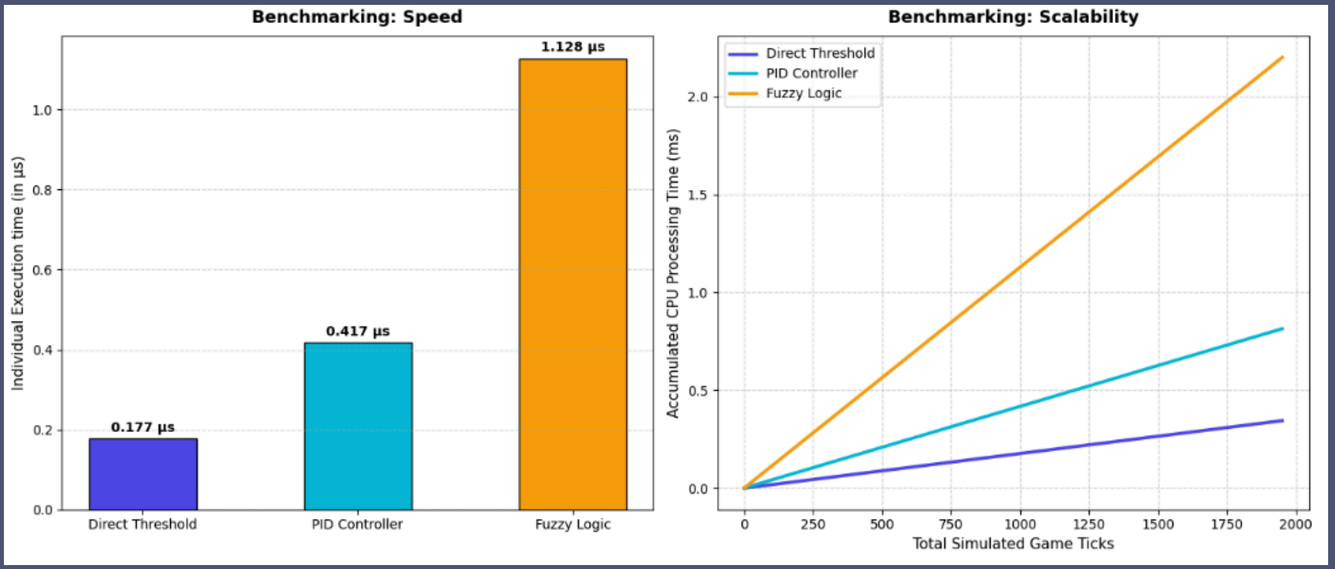
Algorithm: Kahn's Algorithm



BENCHMARKING FOR PROBLEM 1: DYNAMIC DIFFICULTY

As we can see, the DFS-based topological sort is the fastest algorithm, running at 1.1 milliseconds at 1000 nodes. Kahn's Algorithm is the 2nd fastest, almost equal with it on lower upgrade node counts. The execution time in 3.08 milliseconds at 1000 nodes. The Graphlib algorithm serves as a baseline to compare against the two algorithms.

In the end, we chose Kahn's Algorithm, as it's more customizable and allows for easier checking for cyclical errors where a node requires unlocking itself before it becomes accessible, thus making development more error-free and convenient.



BENCHMARKING FOR PROBLEM 2: UPGRADE SYSTEM

As we can see, the benchmarking results show that the fastest algorithm is Direct Threshold with a 0.18 microsecond per individual call, and scales very nicely from the benchmarking chart. However, PID Controller is close with an individual call time of 0.42 microseconds. Fuzzy Logic has the highest call time with 1.13 microseconds.

Ultimately, we have decided to go for the PID Controller in the end because it is similar in speed to the fastest algorithm, but is more dynamic and has a more gradual curve compared to Direct Threshold. Fuzzy Logic is also a plausible candidate, but we find that the complexity of the game isn't that high, and PID strikes a nice middle ground between speed and complexity.

Justification of Algorithm Choices

PID Controller

Time and Space Complexity is O(1), while being more complex and dynamic than the baseline alternative.

From prototype benchmarking code, it performs very fast at 0.4 microseconds while testing with 50000 iterations

Kahn's Algorithm

Time complexity is O(V+E) while Space Complexity is O(V), and strictly limits it to a DAG, thus disallowing cyclical errors.

From prototype benchmarking code, it performs very fast at 3.08 milliseconds while testing with 1000 N (upgrade nodes)

Usage of Algorithms

PID Controller

To create dynamic difficulty, it gathers variables such as the player's base health along with the subwave's enemy budget and currently active enemies.

It will smoothly adjust the next subwave's enemy count based on how many enemies are active and how much health the base has.

Kahn's Algorithm

To create dynamic difficulty, it gathers variables such as the player's base health along with the subwave's enemy budget and currently active enemies.

It will smoothly adjust the next subwave's enemy count based on how many enemies are active and how much health the base has.

```
kp = proportional_gain
ki = integral_gain
kd = derivative_gain

integral_sum = 0
previous_error = 0
error = 0
last_time = get_current_time()

function calculatePID(target_setpoint, current_measured_value):
    current_time = get_current_time()
    dt = current_time - last_time

    # prevents division by zero
    if dt <= 0:
        dt = 0.001

    # calculating error
    error = target_setpoint - current_measured_value

    P = kp * error

    integral_sum = integral_sum + (error * dt)
    I = ki * integral_sum

    derivative = (error - previous_error) / dt
    D = kd * derivative

    output = P + I + D

    previous_error = error
    last_time = current_time

    return output
```

PSEUDOCODE (PID CONTROLLER)

```
function kahnsAlgorithm(graph):
    inDegree = array or map initialized to 0 for all nodes
    queue = empty queue
    topologicalOrder = empty list

    for each node in graph:
        for each neighbor of node:
        inDegree[neighbor] += 1

    for each node in graph:
        if inDegree[node] == 0:
            queue.enqueue(node)

    while queue is not empty:
        currentNode = queue.dequeue()
            topologicalOrder.append(currentNode)

    for each neighbor of currentNode:
        inDegree[neighbor] -= 1

        if inDegree[neighbor] == 0:
            queue.enqueue(neighbor)

    if length(topologicalOrder) != total_number_of_nodes:
        return "Error: Graph contains a cycle (Not a DAG)"

    return topologicalOrder
```

PSEUDOCODE (KAHN'S ALGORITHM)

COMPLEXITY (PID CONTROLLER)

Worst-Case Time Complexity	Best/Average Case Time Complexity	Auxillary Space Complexity
O(1)	O(1)	O(1)

COMPLEXITY (KAHN'S ALGORITHM)

Worst-Case Time Complexity

O(V + E)

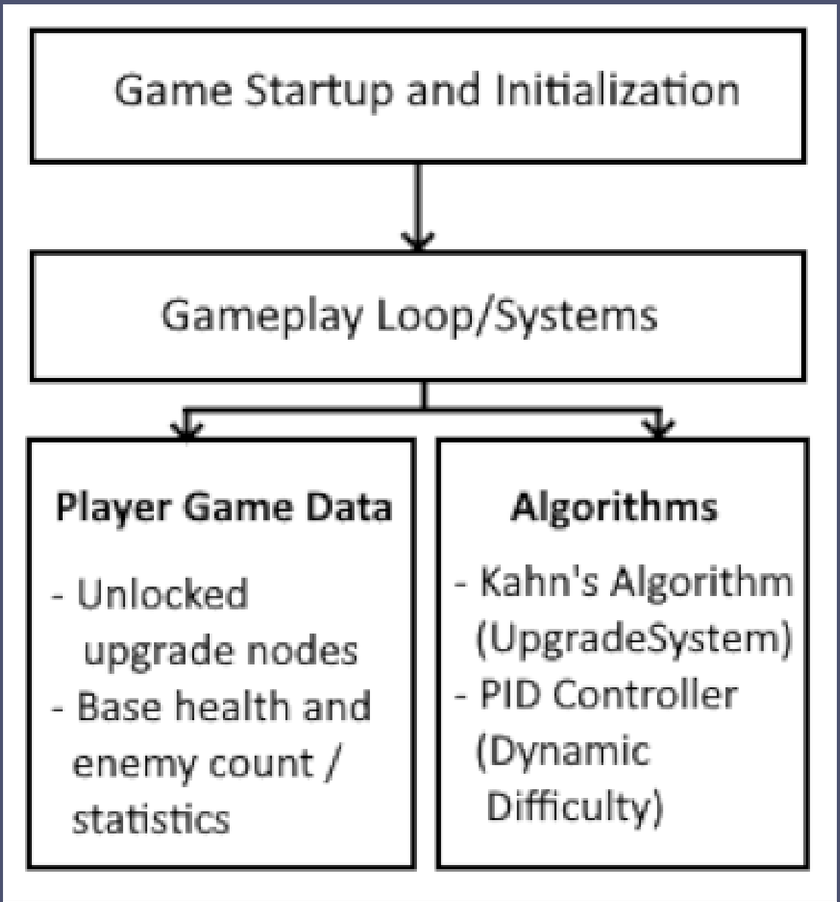
Best/Average Case Time Complexity

O(V + E)

Auxillary Space Complexity

O(V)

ARCHITECTURE COMPONENT DIAGRAM



Layer 1 starts with a **Game Startup and Initialization**, leading to Layer 2 which is the main **Gameplay Loop** as well as its **systems**.

After this, the system accesses the player's **Game Data** (Layer 3) such as the currently unlocked upgrade nodes and the core **Algorithms** (Layer 4), which are the PID Controller and Kahn's Algorithm.

REFERENCES

Aström, K. J., & Hägglund, T. (2005). Advanced PID control. John Wiley & Sons.

Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley.

Hunicke, R. (2005, June). The case for dynamic difficulty adjustment in games. In Proceedings of the 2005 ACM SIGCHI International Conference on Advances in computer entertainment technology (pp. 429-433).

IBM. (2024, November 27). What is content-based filtering?. https://www.ibm.com/think/topics/content-based-filtering

Kahn, A. B. (1962). Topological sorting of large networks. Communications of the ACM, 5(11), 558-562.

Kirkpatrick, S. (1984). Optimization by simulated annealing: Quantitative studies. Journal of statistical physics, 34(5), 975-986.

Vazirani, V. V. (2001). Approximation Algorithms. Springer-Verlag.